

Re-thinking Memory-Bound Limitations in CGRAs

XIANGFENG LIU, Northeastern University, Shenyang, China and City University of Hong Kong, Hong Kong SAR, China

ZHE JIANG, Southeast University, Nanjing, China

ANZHEN ZHU, Northeastern University, Shenyang, China

XIAOMENG HAN, Southeast University, Nanjing, China

MINGSONG LYU, The Hong Kong Polytechnic University, Hong Kong SAR, China

QINGXU DENG, Northeastern University, Shenyang, China

NAN GUAN, City University of Hong Kong, Hong Kong SAR, China

Coarse-Grained Reconfigurable Arrays (CGRAs) are specialized accelerators commonly employed to boost performance in workloads with iterative structures. Existing research typically focuses on compiler or architecture optimizations aimed at improving CGRA performance, energy efficiency, flexibility, and area utilization, under the idealistic assumption that kernels can access all data from Scratchpad Memory (SPM). However, certain complex workloads—particularly in fields like graph analytics, irregular database operations, and specialized forms of high-performance computing (e.g., unstructured mesh simulations)—exhibit irregular memory access patterns that hinder CGRA utilization, sometimes dropping below 1.5%, making the CGRA memory-bound. To address this challenge, we conduct a thorough analysis of the underlying causes of performance degradation, then propose a redesigned memory subsystem and refine the memory model. With both microarchitectural and theoretical optimization, our solution can effectively manage irregular memory accesses through CGRA-specific runahead execution mechanism and cache reconfiguration techniques. Our results demonstrate that we can achieve performance comparable to the original SPM-only system while requiring only 1.27% of the storage size. The runahead execution mechanism achieves an average 3.04× speedup (up to 6.91×), with cache reconfiguration technique providing an additional 6.02% improvement, significantly enhancing CGRA performance for irregular memory access patterns.

CCS Concepts: • **Computer systems organization** → **Reconfigurable computing**; • **Hardware** → **Programmable logic elements**; **Programmable interconnect**; **Hardware accelerators**;

Additional Key Words and Phrases: Coarse-Grained Reconfigurable Array (CGRA), irregular memory access, memory subsystem, runahead execution, cache reconfiguration

We appreciate the reviewers for their insightful and helpful feedback. This work is supported by the National Key Research and Development Program (Grant No. 2024YFB4405600), the National Natural Science Foundation of China (Grant No. 62472086), the Basic Research Program of Jiangsu (Grant No. BK20243042), the Science and Technology Major Special Program of Jiangsu (Grants No. BG2024010), the Start-up Research Fund of Southeast University (Grant No. RF1028624005), and the Provincial Science and Technology Research Project (Grant No. 2024JH2/102400070 and No. 2023JH2/101700370). Authors' Contact Information: Xiangfeng Liu, Northeastern University, Shenyang, Liaoning, China and City University of Hong Kong, Hong Kong SAR, China; e-mail: xiangfengliu.cn@gmail.com; Zhe Jiang (corresponding author), Southeast University, Nanjing, Jiangsu, China; e-mail: zhejiang.uk@gmail.com; Anzhen Zhu, Northeastern University, Shenyang, Liaoning, China; e-mail: j6434858@gmail.com; Xiaomeng Han, Southeast University, Nanjing, Jiangsu, China; e-mail: mingzhihan7@gmail.com; Mingsong Lyu, The Hong Kong Polytechnic University, Hong Kong SAR, China; e-mail: mingsong.lyu@polyu.edu.hk; Qingxu Deng, Northeastern University, Shenyang, Liaoning, China; e-mail: dengqx@mail.neu.edu.cn; Nan Guan, City University of Hong Kong, Hong Kong SAR, China; e-mail: nanguan@cityu.edu.hk. Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1539-9087/2025/09-ART105

<https://doi.org/10.1145/3760386>

ACM Reference Format:

Xiangfeng Liu, Zhe Jiang, Anzhen Zhu, Xiaomeng Han, Mingsong Lyu, Qingxu Deng, and Nan Guan. 2025. Re-thinking Memory-Bound Limitations in CGRAs. *ACM Trans. Embedd. Comput. Syst.* 24, 5s, Article 105 (September 2025), 26 pages. <https://doi.org/10.1145/3760386>

1 Introduction

Recent advancements in science, particularly in AI technologies, increase the demand for accelerated computations. CGRAs consist of configurable networks interconnecting multiple **Processing Elements (PEs)**, each performing computations based on its configuration. Typically, the memory-access PEs can read from or write to a small, low-latency on-chip buffer called SPM. Under ideal conditions, computation-relevant data is fetched from main memory into the SPM before it is used, reducing data-transfer delays and helping sustain high performance by keeping frequently accessed data close to the PEs. Consequently, reliance on high-latency main memory is alleviated. This architecture balances power, performance, and flexibility.

CGRAs accelerate computation-intensive portions of programs (e.g., the one shown in Listing 1), known as kernels, by compiling them into **Data Flow Graphs (DFGs)** and mapping nodes to PEs for parallel execution. The network facilitates data transfer, ensuring throughput and correctness.

Previous research primarily focuses on designing compilers for efficient kernel mapping and developing microarchitectures that enhance CGRA performance [2, 4, 6, 8, 11, 12, 18, 20, 23–26, 38–40, 42, 43, 45]. Most studies assume an idealized memory model where kernel data fits entirely within the SPM, simplifying memory access management. However, irregular and large-volume memory accesses can significantly degrade performance, leading to challenges in real-world deployments.

For workloads, like **Graph Neural Networks (GNNs)** [46], high-performance computing [14], and databases [3], kernels often process large computational data exceeding SPM capacity. These kernels exhibit irregular memory access patterns due to data structure irregularities or algorithmic characteristics, making future accesses unpredictable and prefetching techniques less effective [5, 19]. For example, in GNNs, the irregular connectivity of graph structures results in non-contiguous and unpredictable memory accesses (Figure 1). Arrays accessed depend on values stored in other arrays, leading to indirect access patterns that complicate prefetching (Listing 1) [38].

Under these conditions, our experiments demonstrate that CGRAs encounter stalls when data is absent from the SPM, resulting in utilization dropping as low as 1.43% (Figure 2). Addressing these memory-related bottlenecks is essential for maximizing the acceleration potential of CGRAs. To tackle these challenges, we introduce a comprehensive solution, as illustrated in Figure 3. We redesign the CGRA's memory subsystem by incorporating a cache (Figure 3(a) ⑤). However, irregular accesses lead to poor locality and high cache miss rates. To mitigate this, we introduce a CGRA-specific runahead execution mechanism (Figure 3(b)). Upon a cache miss (①), the CGRA saves its state and enters a Runahead state (②), where valid memory requests trigger precise prefetching (③), loading future required data into the cache (④). This approach enhances cache hit rates, achieving an average speedup of 3.04× (up to 6.91×). To our knowledge, our work is the first in the CGRA domain to explicitly tackle performance degradation from irregular memory access.

As CGRAs scale, multiple PEs sharing a single cache cause severe contention (⑤). We propose a multi-cache design that scales the number of caches proportionally (⑥), effectively isolating cross-PE conflicts. In this multi-cache subsystem, PEs have varying demands for cache configurations. We introduce a hardware/software co-designed cache reconfiguration technique that adjusts cache parameters based on individual PE memory access characteristics (Figure 3(c) ①⑦). This involves reallocating cache ways from PEs with lower demands to those with higher demands (③) and fine-tuning cache line sizes to align with access patterns (④). The contributions include:

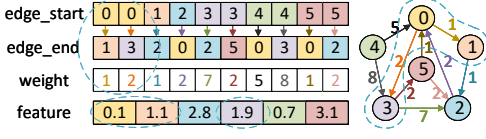


Fig. 1. A graph and its edge list, illustrating the start and end nodes with corresponding edge weights. The *feature* array stores the feature values for each node. The dashed circle in the image highlights the graph's edges, nodes, and their edge list entries.

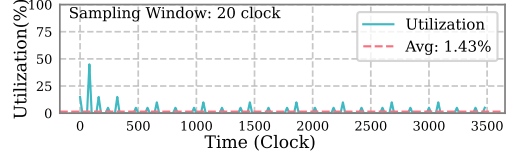


Fig. 2. The absence of data in the SPM causes a significant reduction of CGRA utilization, averaging only 1.43%. Configuration: 4×4 HyCUBE with 4K SPM [18]. Kernel: Feature aggregation in Graph Convolutional Network using the Cora dataset [34].

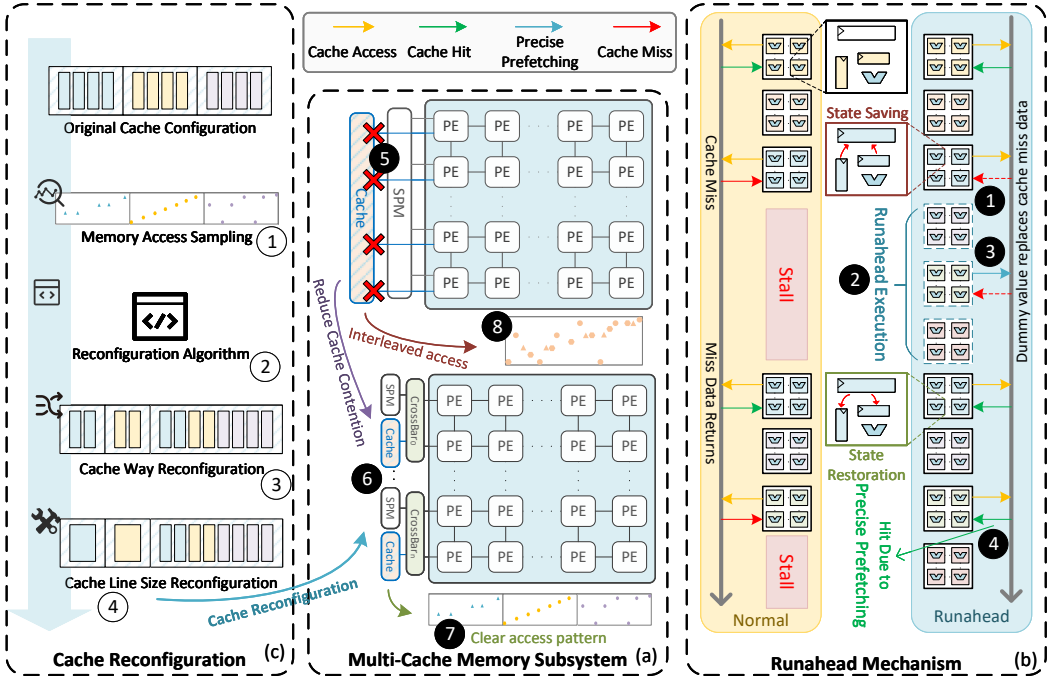


Fig. 3. Overview of the Proposed Architecture: (a) Multi-Cache Memory Subsystem: This design illustrates the evolution of our memory subsystem, which reduces cache contention among PEs and enhances scalability through a multi-cache architecture. (b) Runahead Mechanism: By utilizing CGRA stall times for prefetching, this mechanism reduces cache miss rates and improves CGRA performance. (c) Cache Reconfiguration Process: Cache resources are reallocated and customized according to the specific needs of each PE, optimizing resource utilization and further enhancing overall CGRA performance.

```
for (i = 0; i < X; i++) {
    output_data[edge_start[i]] += weight[i] * feature[edge_end[i]];
}
```

Listing 1. Feature aggregation loop in a GCN. This loop iterates over all edges, applying weights to the features of the destination nodes and aggregating the results into the output data for the source nodes.

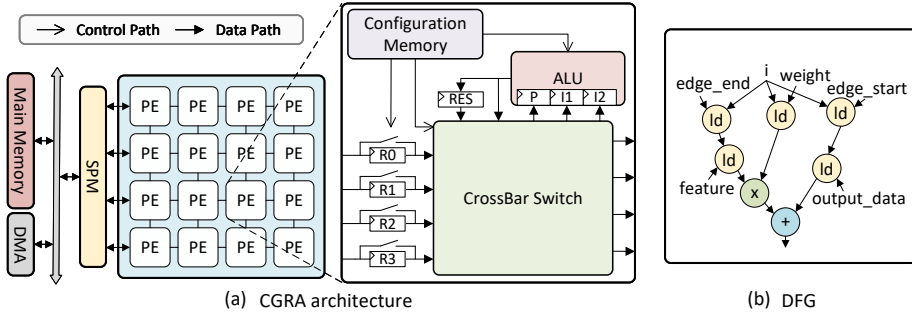


Fig. 4. Overall CGRA architecture and the DFG for listing 1. The CGRA comprises an array of PEs interconnected by a crossbar-based configurable network, with edge PEs connected to an SPM. The SPM communicates with main memory through a bus, and data can also be transferred via DMA (Direct Memory Access). Each PE integrates an ALU, crossbar switch, register file, and configuration memory. To accelerate a kernel, it is transformed into a DFG and then mapped onto the PEs by the mapper.

- **Redesigned CGRA’s Memory Subsystem with Cache Integration:** We address the limitations of traditional SPM-based CGRAs by integrating caches into the memory subsystem, allowing for more efficient handling of large computational data and irregular memory accesses that exceed SPM capacity.
- **CGRA-Specific Runahead Execution Mechanism:** We implement a runahead mechanism for CGRAs, enabling execution during cache misses by substituting dummy values. This significantly improves performance through precise prefetching.
- **Multi-Cache Memory Subsystem:** We develop a highly scalable multi-cache design that dynamically increases cache availability as the CGRA architecture scales, effectively reducing contention in larger architectures and ensuring consistent performance gains.
- **Memory-access-pattern-aware cache reconfiguration:** We introduce cache reconfiguration strategies based on PE memory access patterns. Reallocating cache resources and configuring cache parameters according to PE demands enhances overall performance.

2 Preliminaries

2.1 Microarchitecture of CGRAs

Typical CGRA architecture (Figure 4(a)) [4, 18, 27, 36] consists of an array of PEs interconnected via a crossbar-base configurable network. Edge PEs are connected to an SPM to facilitate load and store operations. Each PE includes an **Arithmetic Logic Unit (ALU)**, a crossbar switch, a register file, and a configuration memory (*config mem*) that controls the behavior of the crossbar switch and ALU on a cycle basis.

To accelerate a kernel, it is transformed into a DFG (Figure 4(b) shows the DFG for listing 1), and a mapper assigns computation nodes to the PEs. This mapping process generates control signals for the ALUs and crossbar switches within the PEs, which are stored in the *config mem*. These control signals ensure that PEs execute operations and manage data transfers according to the DFG.

2.2 Motivation: Memory-Bound Limitations in CGRA

In CGRAs, PEs operate deterministically without handshake signals; all PEs must adhere to the control flow specified in the *config mem* to maintain computational correctness. If a memory-accessing PE cannot retrieve the required data, it causes the entire CGRA to stall. Typically, CGRAs store kernel data in the SPM to ensure that memory-accessing PEs can obtain data as expected.

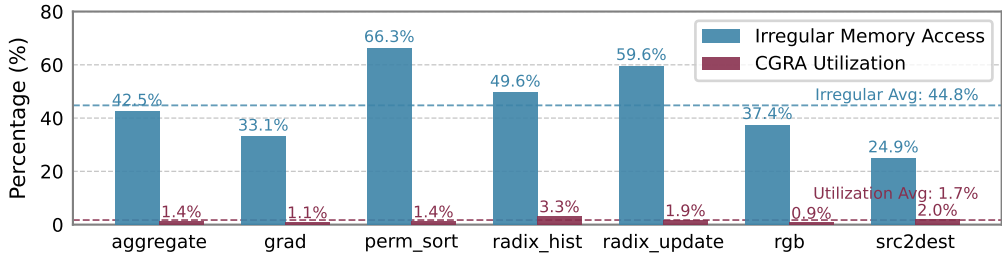


Fig. 5. Proportion of irregular memory access among all memory accesses for various workloads (Table 1), alongside the corresponding CGRA utilization. The findings show that irregular access significantly reduces CGRA utilization to an average of merely 1.7%, thereby having a substantial impact on overall performance.

However, in many real-world kernels, the data required for computation exceeds the capacity of the SPM, and prefetching strategies are effective only for regular memory access patterns.

With irregular memory accesses, necessary data may not reside in the SPM when required, forcing PEs to access main memory. This access can take tens of cycles or more, during which the entire CGRA remains stalled, leading to significant performance degradation due to memory-bound limitations. As illustrated in Figure 2, this issue reduces the average CGRA utilization to as low as 1.43%. Furthermore, Figure 5 quantifies the proportion of irregular memory accesses across various workloads (refer to Table 1) and correlates them with the corresponding CGRA utilization. The results reveal that irregular accesses can reduce the CGRA utilization to an average of just 1.7%, substantially affecting overall performance. Such challenges are common across CGRAs and limit their broader applicability, underscoring the urgent need for architectural optimizations that efficiently handle irregular memory accesses.

To address these limitations, we develop a new memory subsystem for CGRAs designed to mitigate performance degradation caused by irregular and large-volume memory accesses. We implement and evaluate these methods within our simulation framework, demonstrating their effectiveness in enhancing CGRA performance.

Our proposed techniques are broadly applicable to various types of CGRAs. The runahead execution mechanism requires only the addition of state-switching functionality and dummy data tracking capabilities to the base PEs, given that a CGRA comprises multiple instantiated PEs. The cache reconfiguration technique does not necessitate any changes to the overall CGRA architecture.

3 Mitigating Memory-Bound Limitations

This section presents a comprehensive methodology to address the multiple memory-related challenges inherent in CGRAs. Our solution comprises four key components, each targeting specific issues with innovative strategies. In Section 3.1, we redesign the memory subsystem by integrating a cache hierarchy, enabling CGRAs to handle computational data exceeding the capacity of the SPM and to manage irregular memory accesses effectively (Figure 3(a) ⑤). However, our simulations reveal that irregular memory access patterns significantly reduce memory locality, leading to increased cache miss rates and frequent CGRA stalls. To mitigate this issue, Section 3.2 introduces a CGRA-specific runahead execution mechanism (Figure 3(b)). When a cache miss occurs, the CGRA no longer passively stalls; instead, it proactively utilizes the stall time for data prefetching, significantly enhancing cache hit rates and improving overall CGRA performance. As CGRAs scale up, severe contention from sharing a single cache among multiple PEs emerges as a new challenge. Section 3.3 addresses this by proposing a multi-cache architecture that scales the number

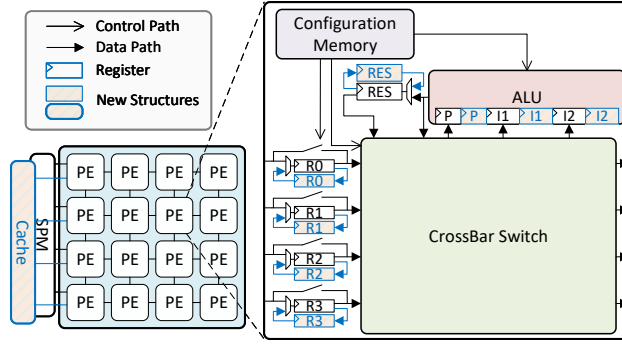


Fig. 6. Integration of cache mechanisms into the CGRA and addition of state save and restore logic.

of caches proportionally with the size of the CGRA, effectively alleviating cache contention in large-scale systems (Figure 3(a) ⑥). Finally, recognizing the unique memory access characteristics and customized cache requirements of different PEs, Section 3.4 develops a cache reconfiguration technique (Figure 3(c)). This dynamic approach reallocates cache resources by shifting cache ways from low-demand to high-demand PEs and adjusts cache line sizes based on the specific memory access characteristics of PEs. This multifaceted approach not only resolves CGRA data capacity limitations but also significantly enhances their ability to process kernels with irregular memory access patterns while ensuring that the memory subsystem scales efficiently with CGRA growth.

3.1 Cache-Integrated Memory Subsystem

As depicted in Figure 6, we develop a new memory subsystem for the CGRA that integrates an SPM with a cache hierarchy comprising L1 and L2 caches. The SPM stores a portion of the computational data, while data exceeding its capacity is accessed through the cache hierarchy. The L1 cache is optimized for low-latency access, and the non-inclusive L2 cache is designed to achieve a higher hit rate. We select this hybrid approach over simply expanding the SPM or entirely replacing it with caches for several reasons:

- **Scalability and Energy Efficiency:** Expanding the SPM to accommodate workloads with irregular memory access patterns results in significant area and power overheads. Caches enhance scalability and energy efficiency, especially for these workloads.
- **Flexibility and Adaptability:** While SPMs are highly efficient for predictable memory access, caches excel at managing irregular accesses. Combining both allows the CGRA to effectively process a broader range of kernels with diverse memory access characteristics.
- **Programming Simplicity:** Incorporating caches reduces the burden on programmers to manually manage data movement, which would be necessary when using SPMs alone.

When the CGRA issues memory access requests, a crossbar switch routes them to either the SPM or the cache. In a single cycle, the CGRA may generate multiple memory access requests; these are arbitrated and sent sequentially to the L1 cache for processing. This design enables the CGRA to handle kernels with data exceeding the SPM capacity and to retrieve data for kernels with irregular memory access patterns via the cache hierarchy.

3.2 Runahead Execution in CGRAs

While our memory subsystem addresses the challenges of handling kernels with data exceeding the capacity of the SPM and irregular memory access patterns, cache misses can still cause the CGRA to stall while waiting for data retrieval, significantly impacting performance. To better utilize stall

time and enhance CGRA efficiency, we draw inspiration from the concept of *runahead execution* in processor design [9]. However, it is important to note that our implementation diverges significantly from traditional processor-based runahead techniques. Unlike CPUs—which have complex control logic, deep pipelines, and speculative execution capabilities—CGRAs represent a fundamentally different architectural paradigm with distributed arrays of PEs that operate with simpler control and deterministic execution.

Our runahead mechanism enables precise prefetching of future data during stall times caused by cache misses. As illustrated in Figure 3(b) ❶, when the CGRA encounters a stall due to a cache miss, it saves its current state and transitions into a runahead state (❷), instead of pausing completely as in normal execution (shown on the left). In this runahead state, the CGRA substitutes missing data with dummy values and continues execution. Concurrently, valid memory access requests trigger precise prefetching (❸), allowing the CGRA to load future required data into the cache as quickly as possible. The system meticulously tracks the propagation of these dummy values; any invalid read or write operations—where either the address or data depends on dummy values—are discarded. To prevent state corruption upon resuming normal execution, valid write operations are redirected to a temporary storage area instead of the cache or SPM.

During runahead execution, write operations are converted into corresponding read operations to trigger prefetching, ensuring no irreversible impact on the system state. Specifically, these writes are never actually committed to memory and serve solely for prefetching purposes, preserving the integrity of the CGRA system state and eliminating the need for rollback upon runahead completion. If invalid data reads occur—where the memory address depends on previously used dummy values—dummy values are used directly. For valid data reads, where the address does not depend on dummy values, the request is sent to the memory subsystem or temporary storage. If the request hits in the SPM, cache, or temporary storage, the retrieved data is used to continue execution. In the case of a cache miss, dummy values are used to allow execution to proceed, while the cache prefetches the relevant data. When the original cache miss that triggered the runahead process is resolved (multiple cache misses may occur, as multiple memory access requests can be issued in a single CGRA cycle), the CGRA restores its previous state and returns to normal execution. Since the necessary data will have already been prefetched during the runahead execution, subsequent cache misses are avoided (❹), preventing further stalls as would occur in normal execution (shown on the left). This approach significantly enhances cache hit rates and reduces stalls.

However, due to the influence of dummy values, not all valid memory requests issued during runahead can be guaranteed to be useful in future execution. For instance, if a memory write operation is incorrectly updated with a dummy value and that data is later read and used, the execution may enter an incorrect state. For example, if a memory write operation does not update data at an address because of dummy values, and this data is subsequently read and utilized, the execution may become inconsistent.

3.2.1 Key Design Aspects of CGRA Runahead Execution. Our CGRA runahead execution mechanism incorporates several critical design elements to enhance performance and ensure data integrity:

- **State Management:** We implement a state save and restore mechanism using dedicated backup registers (Figure 6). This design enables seamless transitions between normal and runahead execution, effectively leveraging idle cycles during cache misses.
- **Dummy Data Handling and Selective Prefetching:** We introduce a dummy data tracking system using additional flag bits and modify existing ALUs to track dummy data propagation. This ensures high-quality, targeted prefetching, significantly reducing cache pollution, and optimizing cache utilization.

- **Temporary Storage Strategy:** To maintain data consistency without discarding potentially useful information, we implement a temporary storage area. After evaluating two approaches, we found that partitioning the SPM is more effective than repurposing existing cache space, as it minimizes cache disruption and provides dedicated space for runahead execution.
- **Cache Policy Optimization:** We configure the cache as non-blocking with a **Least Recently Used (LRU)** replacement policy and a write-allocate strategy. These choices support runahead execution, efficiently manage prefetched data, and enable prefetching for future operations.

Through these considerations, our CGRA runahead mechanism maximizes computational efficiency by utilizing idle cycles during cache misses, maintains data integrity and execution correctness despite the use of runahead execution, and enhances memory subsystem performance while minimizing cache pollution through targeted, high-quality prefetching and sensible cache policy choices. By comprehensively addressing these aspects, our runahead mechanism delivers a substantial enhancement in CGRA performance. This improvement is particularly pronounced for workloads that surpass SPM capacity and are characterized by irregular memory access patterns, ensuring efficient execution and optimal resource utilization.

3.3 Memory Subsystem with Multiple L1 Caches

As the CGRA scales up, the number of memory-accessing PEs increases, leading to potential cache contention when multiple memory accesses are triggered within the same cycle. This contention causes frequent stalls in the CGRA, significantly degrading performance. To address this issue and ensure scalability, we propose a multi-cache memory subsystem comprising multiple L1 caches that share a common L2 cache. As the CGRA grows, the number of L1 caches increases proportionally, maintaining consistent scalability between the memory subsystem and the CGRA.

We opt to partition only the L1 cache for several key reasons. First, since L1 caches are closest to the PEs and have the most significant impact on access latency, partitioning them minimizes contention at this critical level. Second, PEs often exhibit different memory access patterns, and multiple L1 caches can better capture these patterns without interference from other PEs. Third, partitioning only the L1 caches strikes a balance between performance improvement and the additional power associated with cache partitioning. This design choice addresses scalability challenges at the L1 level, where contention is severe, while maintaining the benefits of a shared, larger L2 cache for overall system performance.

Coherence Across Multiple Caches. Although classical consistency protocols can address coherence issues, leveraging CGRA’s unique compilation and architectural features allows us to elegantly and effectively tackle multi-cache coherence in CGRA systems. In our design, each crossbar and its associated physical SPM and cache operate as a single “virtual SPM” (see Figure 3(a) ⑥). Each crossbar is shared by a small number of border PEs (two in Figure 3(a) ⑥), ensuring that each PE directly accesses only its assigned virtual SPM. During compilation, the CGRA’s compile-time data allocation and static scheduling mechanisms partition data among these virtual SPMs according to the computational needs of the connected PEs, ensuring each PE can access its required data. Because the data is fully partitioned across separate virtual SPMs (with no overlap), each cache only handles data from its own partition, thereby eliminating inter-cache consistency conflicts. Within a single virtual SPM, multiple PEs may share the same cache, but their operations follow a statically determined dataflow schedule, preventing potential conflicts over data consistency. Through these design choices, we effectively address multi-cache coherence in our CGRA system.

3.4 Cache Reconfiguration

In our multi-cache memory subsystem, we observe that memory-accessing PEs exhibit unique and distinct memory access patterns, as illustrated in Figure 3(a) ⑦. We categorize these

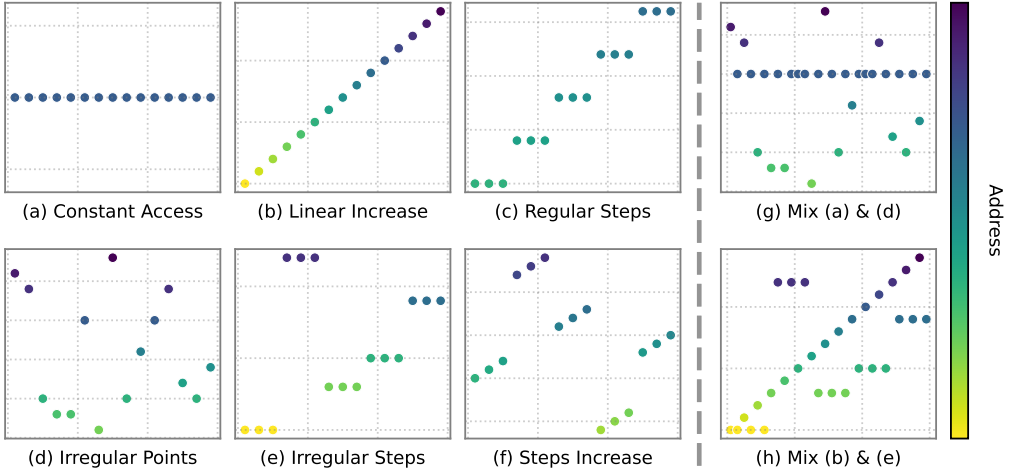


Fig. 7. Memory Access Patterns from Benchmark: Top-left shows regular accesses; bottom-left shows irregular accesses; right shows a mix of both (x-axis: Time, y-axis: Address)

patterns, depicted in Figure 7, into two types: **Regular Memory Accesses**, which show high predictability with strong correlations in adjacent memory addresses—for example, constant addresses (Figure 7(a)), linear increments or decrements (Figure 7(b)), and regular steps (Figure 7(c)); and **Irregular Memory Accesses**, which exhibit low predictability where adjacent accesses do not follow a strong pattern—for example, completely random access points (Figure 7(d)) and irregular steps (Figure 7(e) and 7(f)). Specific patterns may combine elements of both types.

In **Regular Memory Accesses**, the strong correlation between adjacent accesses means that data loaded into the cache is likely to be used in subsequent operations. Therefore, larger cache line sizes are beneficial, effectively prefetching data that will soon be needed. Increasing cache size and associativity provides limited benefits, as data is typically used sequentially and not frequently reused. Cache thrashing may occur between different regular access streams, in which case higher associativity can help mitigate this issue.

In contrast, **Irregular Memory Accesses** lack strong patterns, making it challenging to predict future data needs. Consequently, larger cache sizes are advantageous since they store more data, increasing the likelihood of future cache hits. Higher cache associativity reduces cache thrashing, which is more likely with irregular access patterns compared with regular ones. Cache line size requires careful tuning: too large may lead to prefetching unnecessary data and cause frequent replacements, while too small may not prefetch enough useful data.

These distinct memory access patterns of CGRA memory-accessing PEs are previously obscured when sharing a single cache, as illustrated in Figure 3a ⑧. This phenomenon is analogous to memory accesses in processors where various regular access patterns are interleaved, effectively masking their inherent regularity [33].

The varying cache configuration requirements driven by these diverse memory access patterns, coupled with the variability of CGRA-accelerated kernels, inspire us to propose reconfigurable caches tailored to the memory access patterns exhibited by memory-accessing PEs. As depicted in Figure 8(a), the hardware architecture integrates MMIO-mapped threshold and reconfiguration registers. A dedicated hardware monitor continuously observes cache miss rates, triggering the hardware tracker when a predefined threshold is exceeded. The tracker then samples each PE's memory accesses over a configurable observation window.

- **Issued:** Denotes whether the associated data request has been dispatched to the next-level memory, as some requests might not be issued immediately.

The Load/Store Table, shown in Figure 9(b), records details of each cache miss. Each entry includes:

- **MSHR Entry:** References the corresponding MSHR entry associated with the miss.
- **Dest Reg:** *Read Misses:* Indicates that the miss caused the CGRA to enter runahead mode. When the required data block arrives, it is sent to the CGRA, and normal execution resumes. *Write Misses:* Records the position of the write data in the *Store Buffer*, a dedicated storage unit. Upon data block retrieval, the data is merged with the *Store Buffer* content, written to the cache, and the *Store Buffer* entry is released.
- **Type:** Specifies the instruction type corresponding to the miss (e.g., LW for Load Word, LH for Load Half Word, SW for Store Word), used for data extraction when resolving the miss.

Cache Size Reconfiguration. As shown in Figure 8(b), each cache way is assigned a *permission register* that specifies its associated virtual SPM. By distributing cache ways among different virtual SPMs, we enable reconfiguration of cache size and associativity. Allocating at the cache way level offers two main advantages. First, it adheres to binary address indexing constraints—which require the number of sets to be a power of two—thereby maintaining structural integrity. Second, it simplifies hardware design by allowing cache size adjustments to meet associativity requirements without introducing additional complexity or overhead.

Cache Line Size Reconfiguration. Cache line size reconfiguration involves merging adjacent physical cache lines within a cache way to form a *virtual cache line*. In this context, a traditional cache line is referred to as a physical cache line, while a virtual cache line comprises 2^m physical cache lines, where $m \in \mathbb{Z}_{\geq 0}$ [44].

Cache line replacements occur at the virtual cache line level, involving multiple physical cache lines. However, if a cache access results in a hit, read and write operations continue at the physical cache line granularity. Because the L2 cache line size matches the maximum cache line size of the L1 cache, physical cache lines within a virtual cache line can only result in full hits or full misses, eliminating partial hits. Therefore, treating the virtual cache line as a single unit and processing its physical cache lines sequentially is justified.

For implementing LRU with virtual cache lines, we shift from prioritizing ways within a physical cache set to ways within a virtual cache set. Specifically, the first physical cache set within the virtual cache set serves as the representative. All requests to the virtual cache set, after appropriate masking, are treated as requests to this first physical cache set. When selecting a cache way for LRU replacement, we use the data corresponding to this representative set.

3.4.2 Cache Reconfiguration Strategy. Let A represent the number of L1 caches, S be the total cache size(ways), and L be the set of valid cache line sizes. For $i = 0, 1, \dots, A - 1$, let S_i denote the number of cache ways allocated to the i th L1 cache, and $L_i \in L$ represent its cache line size. The function $h_i(L_i, S_i)$ represents the hit rate of the i th cache with a given cache line size L_i and cache size S_i , while $H_i(S_i) = \max_{L_i} h_i(L_i, S_i)$ denotes the maximum hit rate for the i th cache at cache size S_i .

Our objective is to maximize the sum of the logarithms of $H_i(S_i)$ for all L1 caches:

$$\max_{\{S_i\}} \sum_{i=0}^{A-1} \log H_i(S_i). \quad (1)$$

Subject to the following constraints:

$$\sum_{i=0}^{A-1} S_i \leq S, \quad (2)$$

$$S_i \in \mathbb{Z}_{\geq 0}, \quad \text{for } i = 0, 1, \dots, A - 1. \quad (3)$$

The goal is to allocate cache resources optimally, given the total cache size constraint, in a way that maximizes overall performance. Since a single cache miss in a CGRA can cause the entire array to stall, optimal performance is achieved when all memory accesses result in cache hits during each cycle. Therefore, the objective is to maximize the product of hit rates across all caches.¹ Taking the logarithm of the product converts this into a sum, thereby simplifying the original problem into a linear programming formulation.

To determine the optimal hit rates $H_i(S_i)$ for different cache configurations, we develop a Memory Subsystem Model to simulate cache behavior under different configurations. Using sampled memory access data, we derive the hit rates for each i th L1 cache under various cache configurations.

Using the obtained $H_i(S_i)$, we solve the linear programming problem using Algorithm 1 to find the optimal cache size allocation. Combining the corresponding cache line configurations yields the optimal cache configuration. Figure 10 demonstrates dynamic cache way allocation during scaling by optimally distributing available ways between current/preceding caches under given states.

Improvement: Redefining the Hit Rate. The traditional cache hit rate is typically calculated as:

$$\text{Hit Rate} = \frac{\text{Number of Hits}}{\text{Total Number of Accesses}}$$

This formula calculates the average hit rate, abstracting details such as timing and access patterns. For example, in Figure 7, the Regular and Irregular Access Mixed pattern (Figure 7(g) and 7(h)) shows a significantly higher hit rate than the Irregular Access pattern (Figure 7(d) and 7(e)). Yet, both patterns experience a similar number of misses and CGRA stalls within the same time frame. The hit rate of the Regular and Irregular Mixed pattern (Figure 7(g) and 7(h)) is inflated by frequent regular accesses, leading to diminishing returns as cache size increases.

In our CGRA environment, memory accesses are synchronized across different caches, making it critical to avoid cache misses in all caches within a given time window. Relying on the traditional hit rate for cache allocation may lead to suboptimal cache size assignments, particularly under-allocating for caches with Regular and Irregular Access Mixed patterns (Figure 7(g) and 7(h)), resulting in inefficient gains.

To address this, we redefine the cache miss rate as the **Time Miss Rate**:

$$\text{Time Miss Rate} = \frac{\text{Number of Misses}}{\text{Length of Time Window}}$$

And correspondingly, the **Time Hit Rate** is calculated as $1 - \text{Time Miss Rate}$.

4 Experimental Evaluation

In this section, we evaluate the effectiveness of the proposed methods through a series of experiments. We compare execution outputs both with and without our optimization technique to ensure the reliability of our approach. We implement a cycle-accurate CGRA simulator based on the HyCUBE architecture [18], integrating our redesigned memory subsystem and optimization techniques. Hardware employs varied cache configurations, each specified in respective sections.

¹This is not a strict mathematical condition but serves as a more effective and practical heuristic for improving cache size allocation in synchronized environments.

ALGORITHM 1: Optimal Cache Way Allocation

Function $\text{max_profit}(\mathcal{H}, T_{\max})$ (Time Complexity: $\mathcal{O}(n \cdot T_{\max}^2)$)

Input: $\mathcal{H} \in \mathbb{R}^{n \times (T_{\max}+1)}$: Profit matrix
 $T_{\max} \in \mathbb{N}^+$: Total cache ways available
Output: $(\text{max_profit}, \text{allocations})$: Optimal solution tuple

// Initialization phase
 $n \leftarrow \text{rows}(\mathcal{H})$ *// Number of L1 caches*
Initialize $\text{dp}[0..n][0..T_{\max}] \leftarrow 0$
for $i \leftarrow 1$ **to** n **do**
 $\text{dp}[i][0] \leftarrow \sum_{k=0}^{i-1} \mathcal{H}[k][0]$ *// Base: no allocation*

// DP table construction
for $i \leftarrow 1$ **to** n **do**
 for $j \leftarrow 1$ **to** $T_{\max}$ **do**
 // Default: no allocation to current cache
 $\text{dp}[i][j] \leftarrow \text{dp}[i-1][j] + \mathcal{H}[i-1][0]$
 // Optimal allocation search
 for $k \leftarrow 1$ **to** j **do**
 if $\text{dp}[i-1][j-k] + \mathcal{H}[i-1][k] > \text{dp}[i][j]$
 then
 $\text{dp}[i][j] \leftarrow \text{dp}[i-1][j-k] + \mathcal{H}[i-1][k]$

// Backtrace for optimal allocations
 $\text{allocations} \leftarrow [0]^n$ *// Initialize allocation vector*
 $j \leftarrow T_{\max}$
for $i \leftarrow n$ **downto** 1 **do**
 for $k \leftarrow 0$ **to** j **do**
 if $\text{dp}[i][j] \equiv \text{dp}[i-1][j-k] + \mathcal{H}[i-1][k]$ **then**
 $\text{allocations}[i-1] \leftarrow k$
 $j \leftarrow j - k$
 break

return $(\text{dp}[n][T_{\max}], \text{allocations})$

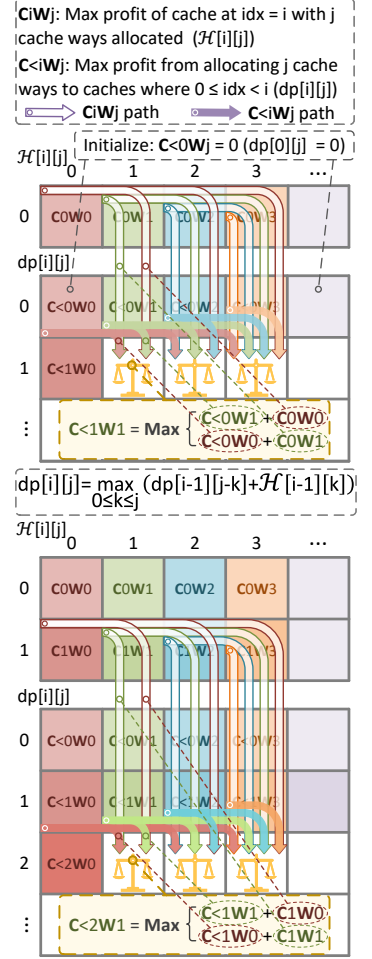


Fig. 10. Illustration of DP table construction (cache count = 1 & 2).

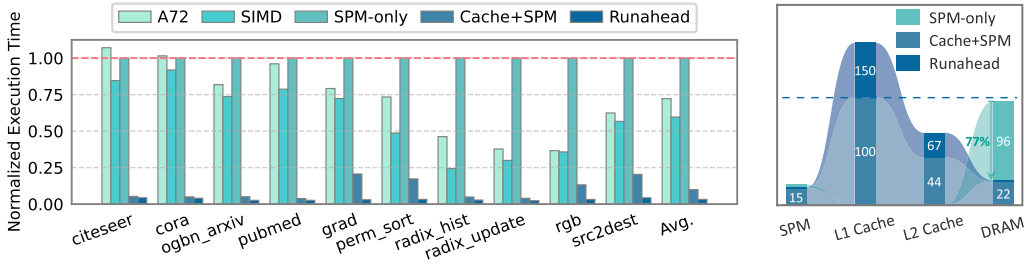
We select HyCUBE as our case study because it is a representative CGRA that effectively illustrates the challenges of this class of architectures, offers a relatively mature and comprehensive end-to-end compilation and simulation framework (albeit not originally cycle-accurate), and is attracting significant research interest.

Our benchmarks cover a diverse range of workloads, including GNN, Computational Fluid Dynamics, Sorting Algorithms, and Image Processing, as listed in Table 1. These benchmarks exhibit both compute-intensive characteristics and irregular memory access patterns, challenging traditional CGRA memory subsystems. They represent a variety of real-world applications where CGRAs can offer significant acceleration and allow us to thoroughly test the effectiveness of our runahead mechanism and cache reconfiguration techniques across diverse computational scenarios.

We use a **Graph Convolutional Network (GCN)** model from PyTorch-Geometric [10], focusing on the aggregate kernel. and select four widely used datasets—Citeseer, Cora, PubMed, and

Table 1. Application Kernels Used in the Evaluation

Application	Kernel	Description	Input Data	Domain
Graph Convolutional Network	aggregate	Aggregates information from neighbors	Citeseer [34] Cora [34] PubMed [34] OGBN-Arxiv [16]	Graph Neural Networks
OpenFOAM	grad	Gradient calculation and correction	Random Data	Computational Fluid Dynamics
Graculus	perm_sort	Counting sort [35]		Graph Clustering
MachSuite [32]	radix_hist	Radix sort (histogram)		Sorting Algorithms
	radix_update	Radix sort (update)		
MiBench [13]	rgb	Converts paletted color to RGB		Image Processing
Berkeley Multimedia Workload [37]	src2dest	Audio processing		Audio Processing



(a) Normalized execution time for ARM Cortex-A72 [1], SIMD [1], SPM-only [18], (b) Normalized statistical Cache-SPM and Runahead. Cache-SPM reduces SPM-related misses and memory access counts of memory components (L1-based). Runahead execution further improves performance.

Fig. 11. Normalized benchmark execution time and memory access statistics.

OGBN-Arxiv—with reduced dimensions to control simulation time. For other kernels, we generate randomized input data to ensure diverse and realistic memory access patterns.

While our benchmark covers a wide range of complex computational patterns, certain advanced operations, especially highly complex graph-based workloads, are limited by existing CGRA compilers. Enhancing these compilers is beyond the scope of this study. In future work, we plan to expand our benchmark suite to include more sophisticated workloads and further explore the potential of our techniques to address these challenges.

4.1 Experimental Results and Analysis

Figure 11(a) compares the execution time of five systems: (1) ARM Cortex-A72 (a high-efficiency core, with configurations detailed in Table 2) [1], (2) SIMD-accelerated A72 utilizing the Neon engine [1], (3) the original HyCUBE design—SPM-only (featuring a 133KB SPM) [18], (4) a Cache+SPM hybrid system with an overall storage capacity matching the SPM-only system (two 512-byte SPM modules, one 4 KB L1 cache, and one 128 KB L2 cache), as specified in Table 3 (base), and (5) a Runahead-enhanced Cache+SPM system.

Prior research focuses on CGRA compilers and cores, leaving SPM as the default choice for data storage due to its simplicity, with limited attention to the memory subsystem. However, our evaluation reveals critical limitations in SPM-only implementations arising from asymmetric access characteristics: while SPM accesses incur near-zero latency, off-SPM memory operations suffer severe penalties. This dichotomy creates a performance degradation—as SPM miss rates increase with CGRA computation scaling, memory latency gradually dominates system behavior, reducing performance to memory-bound levels. Consequently, SPM-only implementations underperform even the A72 and SIMD systems in multiple kernels (Figure 11(a)), indicating that the CGRA’s computational potential is bottlenecked by inefficient data supply from the memory subsystem.

Table 2. A72 and SIMD Configurations [1]

Component	Specification
Core	ARM Cortex-A72 (ARMv8-A) @ 1.8 GHz; Superscalar pipeline (variable-length, OoO); Hybrid BTB/GHB predictor; 128-bit NEON SIMD
L1 Cache	<i>Instruction</i> : 48 KB/core (3-way); <i>Data</i> : 32 KB/core (2-way)
L2 Cache	1 MB shared (16-way)
Memory	8 GB LPDDR4-2400 SDRAM

In contrast, Cache+SPM shows superior scalability by using cache hierarchies. SPM misses are partially offset by cache hits. Benchmarks indicate that Cache+SPM achieves a $10\times$ speedup over the size-equivalent SPM-only design. It also outperforms A72 and SIMD across all kernels, with average speedups of $7.26\times$ and $6.0\times$, respectively. The impact of cache configurations on system performance is analyzed in Section 4.2.

Despite its advantages, Cache+SPM remains susceptible to frequent CGRA stalls caused by irregular memory access patterns. Our runahead mechanism addresses this by proactively prefetching data during CGRA stalls triggered by cache misses. This approach achieves an average $3.04\times$ speedup (up to $6.91\times$) over Cache+SPM, with detailed analysis in Section 4.3.

Figure 11(b) quantifies memory access distribution across SPM-only, Cache+SPM, and Runahead systems. In SPM-only designs, frequent SPM misses force excessive **Dynamic Random Access Memory (DRAM)** accesses (high latency), resulting in low CGRA utilization due to prolonged stalls. Cache+SPM reduces DRAM accesses by 77% by caching frequently reused data, significantly alleviating memory bottlenecks. Runahead optimizes by prefetching data during CGRA stall periods. Compared with SPM+Cache, overall memory subsystem access counts increase, but SPM and DRAM accesses rise less. This is because Runahead mainly activates on L1 cache misses, reducing SPM accesses. Additionally, Runahead's precise prefetching fetches required data from DRAM to cache in advance, limiting DRAM access increases.

In summary, Cache+SPM outperforms SPM-only designs by $10\times$ with 77% fewer DRAM accesses, while runahead prefetching delivers an additional $3.04\times$ speedup (up to $6.91\times$). This highlights the need to integrate cache hierarchies and proactive prefetching to fully exploit CGRA's potential in compute-intensive and irregular memory access workloads.

4.2 Impact of Cache Configuration on CGRA

Based on the configurations in Table 3 (Base), we analyze the effect of each parameter, noting that L1 and L2 cache line sizes adjust together. This experiment aims to establish well-calibrated parameters for a Cache+SPM system, providing a reliable basis for evaluating future optimizations.

Figure 12 illustrates the effects of cache parameter configurations and SPM size on CGRA execution time, and compares the storage requirements of SPM-only and Cache+SPM systems.

In Figure 12(a), the execution time reaches saturation when the L1 cache associativity is set to 8. In some tests, increasing associativity leads to notable performance improvements, primarily due to a reduction in cache thrashing.

Figure 12(b) shows that the execution time saturates when the L1 cache line size reaches approximately 64 bytes. Although the CGRA processes irregular memory accesses during testing, a significant portion of memory accesses remains regular, such as accesses to `edge_end` and `edge_start` in the GCN aggregate kernel. These regular accesses are more sensitive to cache line size, and increasing the cache line size substantially improves the hit rate for these accesses.

In Figure 12(c), increasing the cache size allows more data to be stored for future use, particularly benefiting irregular memory accesses with good locality. However, increasing cache size does not

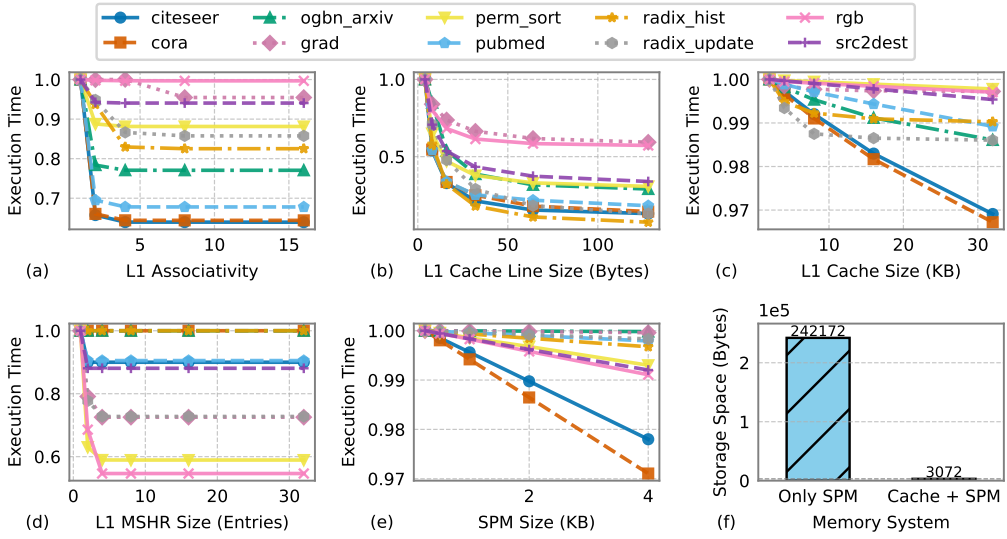


Fig. 12. Execution time is normalized. The cache configuration significantly affects the execution time, with the Cache+SPM system consuming only 1.27% of the storage space compared to the SPM-only system.

always lead to significant performance improvements, as multiple PEs sharing a single L1 cache may cause frequent cache line evictions. Although regular memory accesses show less sensitivity to cache size, they have the potential to displace cache lines utilized by irregular accesses, which can result in performance deterioration.

The **Miss Status Handling Register (MSHR)** limits the number of cache misses that can be handled simultaneously. A CGRA can issue multiple memory requests in a single cycle. In a 4×4 HyCUBE architecture, where memory access occurs on the left PEs, up to four memory access requests can be generated per cycle. If all these requests miss in the cache and the MSHR entries are full, the system must wait for previous requests to complete before accepting new ones. As shown in Figure 12(d), when the MSHR size reaches 4, execution time saturates, as all cache miss requests can be processed concurrently, effectively hiding latency.

Figure 12(e) shows that increasing SPM size has little impact on performance for kernels with large computational data. To analyze storage requirements, we compare SPM-only and Cache+SPM systems using the Cora dataset kernel. Figure 12(f) reveals that Cache+SPM achieves similar performance to SPM-only while using only 1.27% of the size. This conclusion is drawn from a controlled experiment: a Cache+SPM configuration (2 KB L1 cache, 1 KB SPM, 64 B cache line, no L2 cache) is set up and measured. The SPM size in SPM-only is then scaled up until performance matches Cache+SPM. This highlights the memory efficiency gap, emphasizing the limitations of SPM-only.

4.3 Performance Gains from Runahead Execution

Building on the findings from earlier experiments, we employ the relatively optimal Cache+SPM parameter configurations to rigorously evaluate the performance improvements brought by runahead execution. The detailed configurations are provided in Table 3 (Runahead), with a cache line size of 64 B to mitigate potential memory bandwidth pressure caused by larger cache lines.

As shown in Figure 13, incorporating the runahead mechanism results in an average performance improvement of $3.04\times$ (up to $6.91\times$). Tests with a higher degree of randomness show more significant

Table 3. Hardware Configurations (Base vs. Cache+SPM/Runahead vs. Reconfig) [18]

Component/Parameter	Base	Cache+SPM/Runahead	Reconfig
CGRA Architecture	4×4 HyCUBE @ 704 MHz	4×4 HyCUBE @ 704 MHz	8×8 HyCUBE @ 704 MHz
SPM Size	2×512B	2×512B	4×2KB
<i>L1 Cache</i>			
Cache Size/Line Size	4KB/32B	4KB/64B	4×4KB/64B
Associativity	4-way	4-way	8-way
MSHR Entries	16	16	4×16
Hit Latency		1 cycle	
<i>L2 Cache</i>			
Cache Size/Line Size	128KB/32B	128KB/64B	128KB/128B
Associativity		8-way	
Hit/Miss Latency		8/80 cycles	

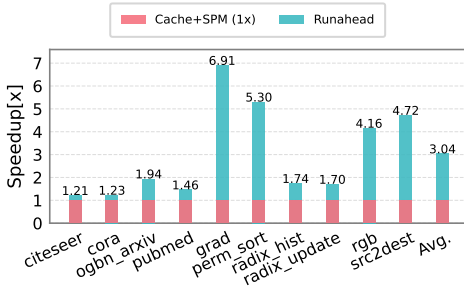


Fig. 13. The runahead mechanism achieves an average performance improvement of 3.04× (up to 6.91×), with particularly significant gains in kernels exhibiting weaker locality.

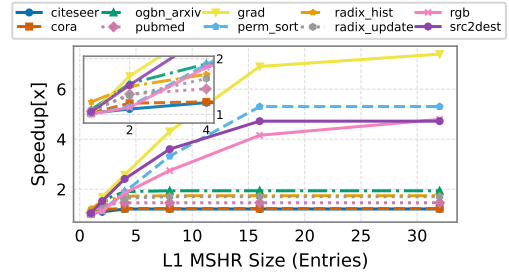


Fig. 14. Kernels with weaker locality require more MSHRs for optimal performance. As the number of MSHRs increases, the benefits of the runahead mechanism gradually grow and saturate around 16.

gains, as they experience more frequent cache misses, allowing runahead execution to substantially reduce the performance penalties associated with these misses, resulting in more significant and measurable performance improvements.

The effectiveness of runahead execution is closely linked to the size of the MSHR. During runahead, a substantial number of cache requests are issued to facilitate prefetching, which relies on the MSHR to store pending cache miss requests. If the MSHR capacity is limited, it constrains the number of prefetches that can be effectively processed during runahead execution. To explore this relationship, we evaluate how the performance speedup from runahead execution varies with different MSHR sizes.

As illustrated in Figure 14, the runahead speedup increases as the MSHR size grows, eventually saturating when the MSHR size reaches approximately 16. Tests with poorer memory locality tend to generate more cache misses, triggering runahead execution more frequently. During these periods, they issue a larger number of cache misses, requiring a greater number of MSHR entries, and as a result, achieve higher speedups from runahead execution. In contrast, tests with better memory locality trigger runahead execution less often, produce fewer cache misses during runahead, and thus require fewer MSHR entries. Consequently, the speedup from runahead execution in these tests is comparatively smaller.

Accuracy. The CGRA runahead mechanism achieves precise prefetching through the meticulous tracking of dummy data, retaining only effective prefetch behaviors. This approach ensures that all

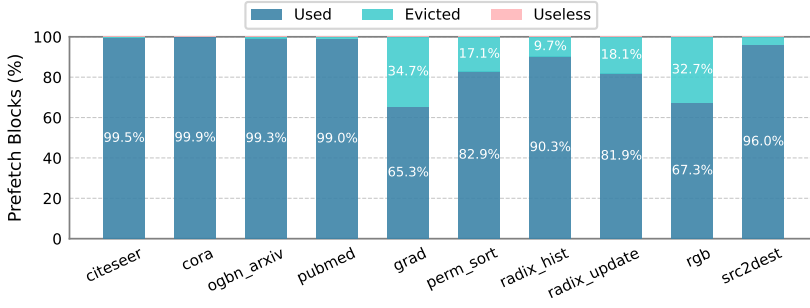


Fig. 15. Distribution of prefetched cache blocks categorized as “Used” (useful blocks successfully utilized by the program), “Evicted” (useful blocks evicted before they can be used), and “Useless” (useless blocks). The near-zero proportion of “Useless” blocks indicates an almost 100% prefetch accuracy. However, in highly random workloads (e.g., grad, rgb), the frequent prefetch and replacement of cache blocks leads to a higher incidence of evicted data, causing misses when the data is eventually needed.

prefetched data will be utilized in future computations, resulting in a prefetch accuracy of nearly 100% for our CGRA runahead mechanism.

Figure 15 illustrates the distribution of cache blocks prefetched during runahead. These cache blocks can be categorized into three groups based on whether they are needed by the program and whether they are successfully used, noting that some prefetched blocks may be evicted before they can be utilized.

The majority of useless blocks (blocks not required by the program) arise under two circumstances. First, near the end of a computation, runahead execution may prefetch data that is never actually accessed. However, this end phase concludes quickly, so the number of unnecessary prefetched cache blocks remains negligible at a global scale. Second, if **Read-After-Write (RAW)** dependencies exist but the corresponding write operation cannot be completed during runahead due to missing data, any subsequent read from that address may fetch incorrect data, thereby driving execution into an invalid state. In such cases, the prefetched data could be useless. Nevertheless, because CGRAs employ a dataflow-based execution model, direct dependencies bypass main memory by traveling across the CGRA’s interconnect network (between PEs), reducing memory accesses and boosting performance and efficiency. Consequently, most RAW dependencies are resolved through the interconnect rather than memory, thereby lowering the risk of incorrect states within our workloads. As a result, the proportion of useless cache blocks is nearly zero, which indirectly confirms that our prefetch strategy achieves close to 100% accuracy.

Nonetheless, we observe that some cache blocks needed by the program are evicted before use. This problem is more pronounced in tests characterized by higher randomness—such as grad and rgb—because frequent cache block prefetches and replacements increase the likelihood of prefetched data being evicted. To address this issue, adjusting relevant cache parameters—such as total size, cache line size, and associativity—to modify how data is mapped to the cache can help reduce conflicts and minimize the eviction of prefetched data.

Coverage. Figure 16 illustrates the coverage, defined as the proportion of memory accesses addressed by the CGRA runahead mechanism. It also depicts the residual portion not mitigated by CGRA runahead prefetching, which manifests as demand misses in the CGRA. The average coverage achieved is 87%. Kernels characterized by poor locality tend to exhibit a higher incidence of misses, presenting greater challenges in prefetching the corresponding data. Consequently, these

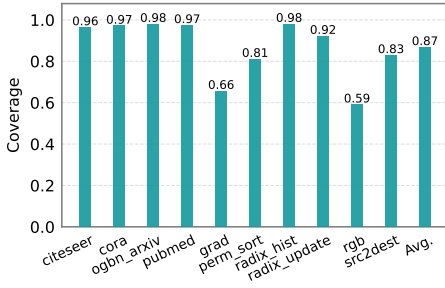


Fig. 16. Percentage of memory accesses covered by CGRA runahead prefetching. Kernels with poor locality exhibit lower coverage due to the heightened difficulty of effective prefetching.

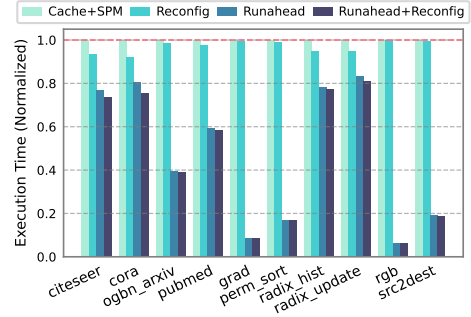


Fig. 17. Cache reconfiguration consistently reduces runtime across various kernels, with those demonstrating higher memory locality achieving more significant performance improvements.

kernels demonstrate relatively lower coverage rates. Notably, a higher coverage does not necessarily imply a greater speedup. This is because in the SPM+Cache system, the prefetching of runahead may overlap with the prefetching of the cache.

4.4 Performance Gains from Cache Reconfiguration

Based on prior experiments, we employ the relatively optimal cache parameter configurations for a rigorous evaluation of cache reconfiguration. The configurations are detailed in Table 3 (Reconfig).

We categorize the kernels into two groups based on the type of input data: real data and randomly generated data. Kernels using real data exhibit irregular memory accesses with some degree of locality, whereas those using randomly generated data generally lack locality. However, certain kernels, such as `radix_hist` and `radix_update`, involve operations like right-shift and bitwise AND on the input data, followed by memory accesses based on the computed results. This process imparts locality to these kernels.

As shown in Figure 17, for kernels with real input data, cache reconfiguration reduces runtime by an average of 4.59% (up to 7.79%) without runahead, and by 3.22% (up to 6.02%) with runahead. For kernels with randomly generated data, cache reconfiguration reduces runtime by an average of 2.10% (up to 5.26%) without runahead, and by 1.58% (up to 2.73%) with runahead.

Since cache misses triggered by irregular memory accesses are the dominant performance bottleneck in CGRAs, our analysis focuses on how cache reconfiguration optimizes such accesses. Without runahead, cache enhancements offer limited benefits for kernels with poor locality, while they can provide more noticeable improvements for kernels with some locality. With runahead, increasing the cache size and adjusting the cache line size can mitigate the issue of prefetched data being prematurely evicted due to the large number of prefetches, leading to more substantial performance improvements for kernels with poor locality. For kernels with a degree of locality, similar cache modifications can also enhance performance.

However, when irregular and regular memory accesses are interleaved, regular accesses may dominate and evict cache lines allocated for irregular accesses, diminishing the benefits of cache reconfiguration (as in `ogbn_arxiv` and `pubmed`).

Based on our comprehensive analysis, we propose two optimization strategies for CGRA compilers. First, optimize memory access node mapping by avoiding the placement of regular and irregular memory accesses on the same PE or PEs sharing the same SPM to prevent interference. Second, optimize memory access timing by adjusting access schedules in shared-cache scenarios

to enable interleaved cache utilization, thereby reducing performance degradation due to cache contention while maintaining overall throughput.

4.5 Hardware Overhead Analysis

We employ a HyCUBE from the CGRA-ME project [4], augmented with context save and restore logic. The HyCUBE supports 32-bit operands and up to 8 contexts. To assess the hardware overhead, we synthesize the modified design using Synopsys Design Compiler in a TSMC 28 nm process. The results indicate that our design incurs an area overhead of 14.78% relative to the native HyCUBE. To avoid the potential influence of varying memory system sizes on our CGRA-optimization overhead analysis, this evaluation focuses solely on the CGRA itself, without involving the memory system.

Since HyCUBE's hardware design is relatively simple, supporting only basic integer operations (e.g., sub, add, multiply, and, or, xor, shl) without division or modulus operations, implementing more complex features—such as floating-point arithmetic—would increase the area of the CGRA. In these scenarios, the relative overhead of our enhancements diminishes proportionally.

The overhead of cache reconfiguration is minimal, comprising two main components: Cache Size Reconfiguration incurs a negligible overhead from additional permission registers assigned to each cache way (e.g., 4-bit permission registers for 32 cache ways add only 128 bits). Cache Line Size Reconfiguration involves managing both the physical cache line associated with a cache miss and other physical cache lines within the same virtual cache line. A counter is added to track processed physical cache lines, but reusing existing mechanisms keeps the overhead minimal.

Overall, the overhead of our design is well justified by the significant performance improvements, especially in handling irregular memory access patterns and processing large computational data efficiently, making the area-performance tradeoff favorable.

4.6 Area Breakdown

Figure 18(a) presents the area breakdown of the system configured as indicated in Table 3 (Reconfig). The L1 Cache occupies 9.38% of the total area, while the 128 KB L2 Cache, sized to accommodate larger data, accounts for 73.32%. Meanwhile, the 8×8 CGRA, featuring relatively simple logic, comprises 12.51% of the area. Figure 18(b) details the CGRA's internal area distribution: the I/O circuitry (for CGRA configuration and memory transactions) occupies 2.99%, while the PE array covers the remaining 97.01%. Within this array, Figure 18(c) shows the breakdown of a single PE. The crossbar, which supports inter-PE configurable interconnect, occupies 27.39% of the PE area. The ALU accounts for 22.10% of the PE area. Figure 18(d) further details the ALU's composition: multiplication logic consumes 52.62%, shift operations (ashr, lshr, shl) account for 23.81%, and the control logic, which governs ALU operations, occupies 9.35%. Bitwise operations (and, or, xor) and comparison require only minimal area.

5 Discussion

In this section, we discuss various aspects of adaptability, scalability, and the CGRA compiler. While our experiments are based on HyCUBE, the proposed optimization techniques are not tied to any unique HyCUBE features but rather emerge from general CGRA design principles, offering strong adaptability. We further analyze the scalability of routing between the multi-cache system and the CGRA, as well as how our other optimization techniques affect the CGRA's area, power, and timing. Finally, we propose a potential path forward for CGRA compilers based on our findings.

5.1 Adaptability

We select HyCUBE as our case study because it is a representative CGRA that effectively illustrates the challenges of this class of architectures and provides a relatively mature, end-to-end compilation

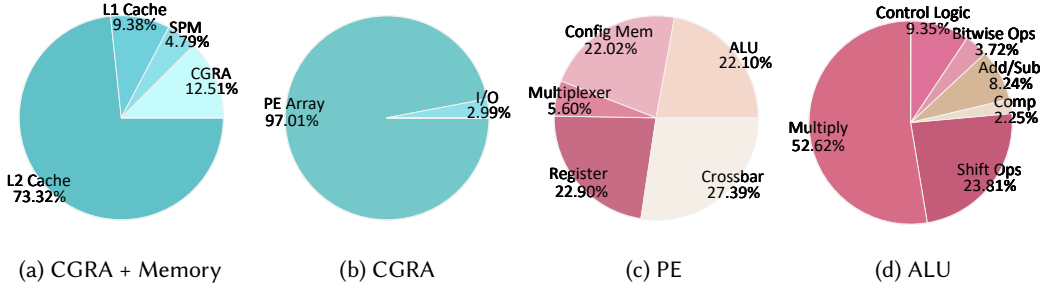


Fig. 18. Area breakdown of the system configured according to Table 3 (Reconfig), including detailed breakdowns for the CGRA, a single PE within the PE array, and the ALU inside that PE.

and simulation framework. Notably, our proposed techniques do not depend on HyCUBE-specific features—such as multi-hop routing—but instead leverage core CGRA design principles [18], thereby demonstrating strong adaptability.

The runahead execution mechanism is triggered when the CGRA stalls due to a cache miss, preserving the current state before runahead execution begins. This approach prefetches future data to reduce cache misses and subsequent stalls. Once runahead execution completes, the saved state is restored, ensuring CGRA correctness. As illustrated in Figure 6, state preservation is achieved by duplicating the relevant register contents in backup registers. Upon entering runahead state, a dedicated runahead signal directs the CGRA to write the current state-related register data into these backup registers. After runahead execution, a multiplexer reconfigures the relevant registers' inputs to the backup register outputs, thereby recovering the saved state. To ensure precise prefetching, dummy data used during runahead are tracked by appending a bit to each datum, indicating whether it is dummy data. The ALU requires only a minor modification—a single OR gate—to propagate this bit from the input operands to the output. These additions operate in parallel with the original logic and remain decoupled, thereby minimizing disruptions to the existing design. Moreover, they do not rely on any HyCUBE-specific features but instead align with common CGRA design principles. Because CGRAs typically consist of multiple uniform PEs, this single modification can simply be replicated across all PEs, further streamlining the implementation.

Since the cache reconfiguration technique operates as a separate memory subsystem optimization, it remains decoupled from the CGRA architecture and thus can be readily extended to other CGRAs. As a result, our proposed approaches are broadly applicable to a diverse range of CGRA designs.

5.2 Scalability

In this section, we analyze the proposed CGRA system's scalability with respect to four key dimensions: routing, area, power, and timing. Our results show that the design maintains linear complexity in each dimension, as described below.

Routing Scalability. For the routing between the CGRA and the memory subsystem, as illustrated in Figure 3(a) ⑥, two PEs share a single cache and SPM pair via a crossbar, which obviates the need for direct connectivity between every PE and every cache/SPM block. Consequently, for an $n \times n$ CGRA, the routing complexity grows proportionally to $2n$. Within the CGRA itself, the CGRA features a regular array structure in which each PE connects only to nearby PEs within a certain distance. Our optimization does not modify the underlying CGRA interconnect, thus preserving the original linear complexity as the CGRA scales.

Area/power Scalability. Because the CGRA’s regular, array-based architecture inherently scales linearly with the number of PEs, our modifications—which preserve this fundamental layout—maintain its linear area scalability. Furthermore, in array-based CGRA designs with consistent voltage and frequency, power consumption tends to increase proportionally to area; consequently, as the CGRA’s area grows linearly, its power consumption follows a similar linear trend.

Timing Scalability. The CGRA’s critical path typically resides in the data path from a PE’s output register to a successive PE’s input register or within the PE’s ALU logic [18]. As discussed in Section 5.1 and illustrated in Figure 6, the introduced enhancements (e.g., state save/restore logic, dummy data tracking) are logically simple and remain decoupled from the original CGRA data paths, thereby neither extending existing critical paths nor introducing new ones. Consequently, the critical path remains effectively unchanged as the CGRA scales.

5.3 CGRA Compiler

Although current CGRA compilers excel at mapping and scheduling well-structured loops efficiently, challenges remain—particularly in accommodating complex control flows, dynamic loops, irregular data structures, and memory-aware scheduling. This section proposes several potential directions to advance CGRA compiler capabilities.

A key avenue lies in leveraging **Multi-Level Intermediate Representation (MLIR)** [21]. Its comprehensive transformation passes readily handle intricate constructs, while preserving domain-specific abstractions. The dialect system also supports custom operations tailored for CGRAs, enabling the representation of diverse elements—from complex DFGs to various memory access patterns—under a single IR (Intermediate Representation) [22].

Integrating **Large Language Models (LLMs)** into compiler toolchains marks another promising direction [7, 15]. Coupled with their understanding of code, this capability could inform transformation passes, suggest optimized rewrites, and refine pass ordering. By addressing challenges such as pointer aliasing and irregular loop constructs, LLM-guided optimizations show significant potential for tackling complex compilation scenarios.

Equally pivotal is memory-aware scheduling. Traditional methods often focus on the compute graph alone, overlooking memory contention. Incorporating explicit memory modeling into MLIR ensures that every pass accounts for memory hierarchies, from shared caches to specialized SPMs.

Finally, unified abstractions that encompass computation and memory can bolster the synergy between scheduling and resource allocation. By representing both computational kernels and their memory requirements in a single IR, compilers can more effectively perform mapping, distinguishing predictable, regular operations from dynamic, irregular ones to mitigate cache conflicts.

In summary, a holistic strategy integrating MLIR’s flexible IR, LLM-guided optimizations, memory-aware scheduling, and unified abstractions of computation and memory charts a potential path forward for CGRA compiler evolution.

6 Related Work

Cheng Tan et al. [38] propose a CGRA architecture that dynamically balances its pipeline by adjusting resource allocation in response to variations in kernel execution times, thereby improving throughput in streaming applications. Their evaluation, focused on GCN applications using a small dataset that fits entirely within the SPM, primarily examines execution time variations due to input data changes. Consequently, their study does not address challenges related to SPM capacity limitations or issues arising from prefetching in kernels with irregular memory access patterns.

CASCADE [41] presents a decoupled access-execute CGRA design that separates **address generation instructions (AGIs)** from the primary computation instructions, thereby alleviating the

on-array overhead and improving loop kernel performance. Its dedicated hardware, referred to as a “stream engine,” handles the memory address generation for typical strided access patterns. However, while CASCADE handles regular strided accesses well, it can face challenges when dealing with irregular memory patterns that strain the stream engine’s capabilities. To address these scenarios, our work introduces a runahead-based prefetching scheme that efficiently mitigates irregular memory stalls by looking ahead to future accesses, regardless of their stride regularity. We supplement this approach with a reconfigurable cache subsystem, adjusting the caching strategy for different memory behavior profiles. By tailoring cache configurations per PE, our system accommodates a broader range of access patterns while preserving high computational efficiency.

Runahead execution [9, 28–31] allows processors to continue executing future instructions when stalled due to long-latency memory accesses, enabling data prefetching and partially hiding memory latencies. While implementing runahead in processors is challenging due to their complex hardware designs and variable execution logic, CGRAs have simpler, more regular architectures by offloading computational complexity to the compiler. This simplicity facilitates the implementation of runahead in CGRAs, allowing them to more readily leverage its benefits.

Previous cache optimization efforts, like the self-adjusting cache architecture by Chuanjun Zhang et al. [44] and the *Hopscotch* framework by Zhe Jiang et al. [17], focus on traditional CPU architectures. These studies optimize cache parameters or dynamically allocate cache capacity to improve performance and power consumption. However, CGRAs present unique challenges due to their fundamentally different microarchitectures. Implementing runahead execution in CGRAs requires not only non-blocking caches but also customized cache adaptations to match CGRA-specific characteristics—an aspect not typically addressed in traditional cache studies. In CPU architectures with interleaved regular access patterns, the inherent regularity is obscured, complicating efforts to leverage them for optimization. In contrast, CGRAs expose these distinct memory access patterns, unveiling novel and unique optimization opportunities. Our work addresses these challenges and harnesses new optimization opportunities by designing customized cache architectures and developing dynamic cache reconfiguration techniques. These techniques are tailored to better support CGRA runahead execution while also adapting to the unique memory access patterns of CGRA workloads, thereby enhancing cache utilization and significantly improving performance.

7 Conclusion

Modern workloads in domains such as Graph Neural Networks, high-performance computing, and databases feature numerous compute-intensive kernels well-suited for acceleration using CGRAs. However, many of these kernels exhibit irregular memory access patterns, rendering traditional CGRA systems based on SPM inadequate. Frequent memory misses cause CGRAs to become memory-bound, significantly limiting their broader adoption. To address this challenge, we design a memory subsystem for CGRAs that integrates cache mechanisms and implement a *runahead execution* technique specifically tailored for CGRAs, resulting in substantial performance improvements. To ensure scalability, we introduce a *multi-cache memory subsystem*, enabling the memory architecture to scale consistently with the CGRA. Furthermore, we observe that memory-accessing PEs in CGRAs exhibit unique memory access patterns, requiring personalized cache configurations. To meet these requirements and optimize cache utilization, we implement *cache size reconfiguration* across L1 caches based on cache ways and *cache line size reconfiguration* within each cache. Our comprehensive methodology addresses a critical bottleneck in CGRA design, dramatically expanding their computational scope to encompass a wider spectrum of applications, including those previously bounded by memory subsystem constraints. The proposed techniques not only augment CGRA functionality but also provide valuable heuristics for optimizing memory hierarchies in diverse accelerator architectures.

References

- [1] Arm Limited. 2025. Documentation —Arm Developer. Retrieved from <https://developer.arm.com/documentation/100095/0003/Introduction/Features>. (2025). [Accessed 29-03-2025].
- [2] Thilini Kaushalya Bandara, Dhananjaya Wijerathne, Tulika Mitra, and Li-Shiuan Peh. 2022. REVAMP: A Systematic Framework for Heterogeneous CGRA Realization. (2022). DOI: <https://doi.org/10.5281/zenodo.5848404>
- [3] Spyros Blanas, Yinan Li, and Jignesh M. Patel. 2011. Design and evaluation of main memory hash join algorithms for multi-core CPUs. In *Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data (SIGMOD '11)*. Association for Computing Machinery, New York, NY, USA, 37–48. DOI: <https://doi.org/10.1145/1989323.1989328>
- [4] S. Alexander Chin, Noriaki Sakamoto, Allan Rui, Jim Zhao, Jin Hee Kim, Yuko Hara-Azumi, and Jason Anderson. 2017. CGRA-ME: A unified framework for CGRA modelling and exploration. In *2017 IEEE 28th International Conference on Application-Specific Systems, Architectures and Processors (ASAP)*. 184–189. DOI: <https://doi.org/10.1109/ASAP.2017.7995277>
- [5] Jason Cong, Hui Huang, Chunyue Liu, and Yi Zou. 2011. A reuse-aware prefetching scheme for scratchpad memory. In *2011 48th ACM/EDAC/IEEE Design Automation Conference (DAC)*. 960–965.
- [6] Jason Cong, Hui Huang, Chiyuan Ma, Bingjun Xiao, and Peipei Zhou. 2014. A fully pipelined and dynamically composable architecture of CGRA. In *2014 IEEE 22nd Annual International Symposium on Field-Programmable Custom Computing Machines*. 9–16. DOI: <https://doi.org/10.1109/FCCM.2014.12>
- [7] Chris Cummins, Volker Seeker, Dejan Grubisic, Mostafa Elhoushi, Youwei Liang, Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Kim Hazelwood, Gabriel Synnaeve, and Hugh Leather. 2023. Large Language Models for Compiler Optimization. (2023). arXiv:2309.07062. Retrieved from <https://arxiv.org/abs/2309.07062>
- [8] Shail Dave and Aviral Shrivastava. 2018. Ccf: A cgra compilation framework. In *Proc. 21st Design Autom. Test Europe (DATE)*. 1.
- [9] James Dundas and Trevor Mudge. 1997. Improving data cache performance by pre-executing instructions under a cache miss. In *Proceedings of the 11th International Conference on Supercomputing (ICS '97)*. Association for Computing Machinery, New York, NY, USA, 68–75. DOI: <https://doi.org/10.1145/263580.263597>
- [10] Matthias Fey and Jan Eric Lenssen. 2019. Fast graph representation learning with PyTorch Geometric. arXiv:1903.02428. Retrieved from <https://arxiv.org/abs/1903.02428>
- [11] Graham Gobieski, Ahmet Oguz Atli, Kenneth Mai, Brandon Lucia, and Nathan Beckmann. 2021. Snafu: An ultra-low-power, energy-minimal cgra-generation framework and architecture. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1027–1040.
- [12] Yijiang Guo and Guojie Luo. 2020. Pillars: An integrated CGRA design framework. In *Third Workshop on Open-Source EDA Technology (WOSET)*. 1–5.
- [13] M.R. Guthaus, J.S. Ringenberg, D. Ernst, T.M. Austin, T. Mudge, and R.B. Brown. 2001. MiBench: A free, commercially representative embedded benchmark suite. In *Proceedings of the Fourth Annual IEEE International Workshop on Workload Characterization. WWC-4 (Cat. No.01EX538)*. 3–14. DOI: <https://doi.org/10.1109/WWC.2001.990739>
- [14] Michael Allen Heroux and Jack Dongarra. 2013. *Toward a New Metric for Ranking High Performance Computing Systems*. Technical Report. Sandia National Lab.(SNL-NM), Albuquerque, NM (United States); University of...
- [15] Charles Hong, Sahil Bhatia, Alvin Cheung, and Yakun Sophia Shao. 2025. Autocomp: LLM-Driven Code Optimization for Tensor Accelerators. (2025). arXiv:2505.18574. Retrieved from <https://arxiv.org/abs/2505.18574>
- [16] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 1855, 16 pages.
- [17] Zhe Jiang, Kecheng Yang, Nathan Fisher, Nan Guan, Neil C. Audsley, and Zheng Dong. 2024. Hopscotch: A hardware-software co-design for efficient cache resizing on multi-core SoCs. *IEEE Transactions on Parallel and Distributed Systems* 35, 1 (2024), 89–104. DOI: <https://doi.org/10.1109/TPDS.2023.3332711>
- [18] Manupa Karunaratne, Aditi Kulkarni Mohite, Tulika Mitra, and Li-Shiuan Peh. 2017. HyCUBE: A CGRA with reconfigurable single-cycle multi-hop interconnect. In *2017 54th ACM/EDAC/IEEE Design Automation Conference (DAC)*. 1–6. DOI: <https://doi.org/10.1145/3061639.3062262>
- [19] Youngbin Kim, Kyoungwoo Lee, and Aviral Shrivastava. 2019. Static function prefetching for efficient code management on scratchpad memory. In *2019 IEEE 37th International Conference on Computer Design (ICCD)*. 350–358. DOI: <https://doi.org/10.1109/ICCD46524.2019.00056>
- [20] Mingyang Kou, Jiangyuan Gu, Shaojun Wei, Hailong Yao, and Shouyi Yin. 2020. TAEM: Fast transfer-aware effective loop mapping for heterogeneous resources on CGRA. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. 1–6. DOI: <https://doi.org/10.1109/DAC18072.2020.9218668>
- [21] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling compiler infrastructure for domain

- specific computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 2–14. DOI : <https://doi.org/10.1109/CGO51591.2021.9370308>
- [22] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2020. MLIR: A compiler infrastructure for the end of Moore’s law. (2020). arXiv:2002.11054. Retrieved from <https://arxiv.org/abs/2002.11054>
 - [23] Hongsik Lee, Dong Nguyen, and Jongeun Lee. 2015. Optimizing stream program performance on CGRA-based systems?. In *2015 52nd ACM/EDAC/IEEE Design Automation Conference (DAC)*. 1–6. DOI : <https://doi.org/10.1145/2744769.2744884>
 - [24] Zhaoying Li, Dhananjaya Wijerathne, Xianzhang Chen, Anuj Pathania, and Tulika Mitra. 2022. ChordMap: Automated mapping of streaming applications onto CGRA. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 2 (2022), 306–319. DOI : <https://doi.org/10.1109/TCAD.2021.3058313>
 - [25] Dajiang Liu, Yuxin Xia, Jiaxing Shang, Jiang Zhong, Peng Ouyang, and Shouyi Yin. 2024. E2EMap: End-to-end reinforcement learning for CGRA compilation via reverse mapping. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 46–60. DOI : <https://doi.org/10.1109/HPCA57654.2024.00015>
 - [26] Yixuan Luo, Cheng Tan, Nicolas Bohm Agostini, Ang Li, Antonino Tumeo, Nirav Dave, and Tong Geng. 2023. ML-CGRA: An integrated compilation framework to enable efficient machine learning acceleration on CGRAs. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. 1–6. DOI : <https://doi.org/10.1109/DAC56929.2023.10247873>
 - [27] Bingfeng Mei, Serge Vernalde, Diederik Verkest, Hugo De Man, and Rudy Lauwereins. 2003. ADRES: An architecture with tightly coupled VLIW processor and coarse-grained reconfigurable matrix. In *International Conference on Field-Programmable Logic and Applications*. Retrieved from <https://api.semanticscholar.org/CorpusID:39182312>
 - [28] O. Mutlu, Hyesoon Kim, J. Stark, and Y.N. Patt. 2005. On reusing the results of pre-executed instructions in a runahead execution processor. *IEEE Computer Architecture Letters* 4, 1 (2005), 2–2. DOI : <https://doi.org/10.1109/L-CA.2005.1>
 - [29] O. Mutlu, J. Stark, C. Wilkerson, and Y.N. Patt. 2003. Runahead execution: An alternative to very large instruction windows for out-of-order processors. In *The Ninth International Symposium on High-Performance Computer Architecture, 2003. HPCA-9 2003. Proceedings*. 129–140. DOI : <https://doi.org/10.1109/HPCA.2003.1183532>
 - [30] Ajeya Naithani, Sam Ainsworth, Timothy M. Jones, and Lieven Eeckhout. 2021. Vector runahead. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 195–208. DOI : <https://doi.org/10.1109/ISCA52012.2021.00024>
 - [31] Ajeya Naithani, Jaime Roelandts, Sam Ainsworth, Timothy M. Jones, and Lieven Eeckhout. 2023. Decoupled vector runahead. In *2023 56th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 17–31.
 - [32] Brandon Reagen, Robert Adolf, Yakun Sophia Shao, Gu-Yeon Wei, and David Brooks. 2014. MachSuite: Benchmarks for accelerator design and customized architectures. In *2014 IEEE International Symposium on Workload Characterization (IISWC)*. 110–119. DOI : <https://doi.org/10.1109/IISWC.2014.6983050>
 - [33] Reza Salkhordeh, Shahriar Ebrahimi, and Hossein Asadi. 2018. ReCA: An efficient reconfigurable cache architecture for storage systems with online workload characterization. *IEEE Transactions on Parallel and Distributed Systems* 29, 7 (2018), 1605–1620. DOI : <https://doi.org/10.1109/TPDS.2018.2796100>
 - [34] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Gallagher, and Tina Eliassi-Rad. 2008. Collective classification in network data. In *The AI Magazine*. Retrieved from <https://api.semanticscholar.org/CorpusID:62016134>
 - [35] Harold Herbert Seward. 1954. *Information sorting in the application of electronic digital computers to business operations*. Ph.D. Dissertation. Massachusetts Institute of Technology. Department of Electrical Engineering.
 - [36] Hartej Singh, Ming-Hau Lee, Guangming Lu, Nader Bagherzadeh, Fadi J. Kurdahi, and Eliseu M. Chaves Filho. 2000. MorphoSys: An integrated reconfigurable system for data-parallel and computation-intensive applications. *IEEE Trans. Comput.* 49, 5 (may 2000), 465–481. DOI : <https://doi.org/10.1109/12.859540>
 - [37] Nathan T. Slingerland and Alan Jay Smith. 2002. Design and characterization of the Berkeley multimedia workload. *Multimedia Systems* 8, 4 (2002), 315–327.
 - [38] Cheng Tan, Nicolas Bohm Agostini, Tong Geng, Chenhao Xie, Jiajia Li, Ang Li, Kevin J. Barker, and Antonino Tumeo. 2022. DRIPS: Dynamic rebalancing of pipelined streaming applications on CGRAs. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 304–316. DOI : <https://doi.org/10.1109/HPCA53966.2022.00030>
 - [39] Cheng Tan, Chenhao Xie, Ang Li, Kevin J. Barker, and Antonino Tumeo. 2020. OpenCGRA: An open-source unified framework for modeling, testing, and evaluating CGRAs. In *2020 IEEE 38th International Conference on Computer Design (ICCD)*. 381–388. DOI : <https://doi.org/10.1109/ICCD50377.2020.00070>
 - [40] Christopher Torng, Peitian Pan, Yanghui Ou, Cheng Tan, and Christopher Batten. 2021. Ultra-Elastic CGRAs for irregular loop specialization. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 412–425. DOI : <https://doi.org/10.1109/HPCA51647.2021.00042>
 - [41] Dhananjaya Wijerathne, Zhaoying Li, Manupa Karunarathne, Anuj Pathania, and Tulika Mitra. 2019. CASCADE: High Throughput Data Streaming via Decoupled Access-Execute CGRA. *ACM Trans. Embed. Comput. Syst.* 18, 5s, Article 50 (Oct. 2019), 26 pages. DOI : <https://doi.org/10.1145/3358177>

- [42] Dhananjaya Wijerathne, Zhaoying Li, Manupa Karunaratne, Li-Shiuan Peh, and Tulika Mitra. 2022. Morpher: An open-source integrated compilation and simulation framework for cgra. In *Fifth Workshop on Open-Source EDA Technology (WOSET)*.
- [43] Jackson Woodruff, Thomas Koehler, Alexander Brauckmann, Chris Cummins, Sam Ainsworth, and Michael F. P. O’Boyle. 2023. Rewriting history: Repurposing domain-specific CGRAs. arXiv:2309.09112. Retrieved from <https://arxiv.org/abs/2309.09112>
- [44] Chuanjun Zhang, Frank Vahid, and Roman Lysecky. 2004. A self-tuning cache architecture for embedded systems. *ACM Trans. Embed. Comput. Syst.* 3, 2 (may 2004), 407–425. DOI : <https://doi.org/10.1145/993396.993405>
- [45] Zhongyuan Zhao, Weiguang Sheng, Qin Wang, Wenzhi Yin, Pengfei Ye, Jinchao Li, and Zhigang Mao. 2020. Towards higher performance and robust compilation for CGRA modulo scheduling. *IEEE Transactions on Parallel and Distributed Systems* 31, 9 (2020), 2201–2219. DOI : <https://doi.org/10.1109/TPDS.2020.2989149>
- [46] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. 2020. Graph neural networks: A review of methods and applications. *AI Open* 1 (2020), 57–81. DOI : <https://doi.org/10.1016/j.aiopen.2021.01.001>

Received 7 August 2025; revised 7 August 2025; accepted 7 August 2025